

Application Report — Prefects Discipline

Generated: August 10, 2026 · Source: full code review + live probe of the production Supabase database.

1. Overview

Prefects Discipline is a school prefect discipline-management dashboard. Prefects search a student roster, record disciplinary actions (strikes, black marks), positive records (gold marks), punishments, and free-text comments, browse all records, group students into named lists, and export a daily PDF report.

The application is a **100% client-side Next.js App Router SPA** — there are no server API routes, no server components that fetch data, and no backend of its own. Every page talks directly to a hosted **Supabase (Postgres)** database through the Supabase JavaScript client using a publishable key embedded in the bundle.

Primary flows

Flow	How it works
Login	/authenticate form → queries the users table → verifies bcrypt hash → stores the user in localStorage
Search a student	Admission No exact match, or a normalized client-side substring search on the name
View a student	/student/[admissionNo] detail page with counts + history per record type

Flow	How it works
Add a record	Modal form on dashboard / detail page → supabase.from(...).insert(...) directly from the browser
Browse records	/discipline ("Records") page: unified chronological feed across all 5 record types, tabs, filters, sorting
Manage lists	/lists admin-only area: create lists, add/remove students, bulk-assign strikes/black marks
Daily report	Admin-only header button → generates a PDF of today's strikes + black marks, auto-downloads
Manage users	/users admin-only page: add / edit / delete system users with roles

2. Tech Stack

Layer	Technology	Version
Framework	Next.js (App Router)	16.2.10
UI library	React	19.2.4
Language	TypeScript	5.x
Styling	Tailwind CSS (v4, via @tailwindcss/postcss)	4.x
Backend / Database	Supabase (hosted Postgres + PostgREST)	supabase-js 2.110.3
Password hashing	bcryptjs	3.0.3
PDF generation	jspdf + jspdf-autotable	4.2.1 / 5.0.8
Linting	ESLint (eslint-config-next)	9.x

Scripts: npm run dev, npm run build, npm run start, npm run lint.

3. Architecture & How It Works

Browser (React client components)

|

| @supabase/supabase-js (publishable key, embedded in bundle)

▼

Supabase PostgREST REST API → Postgres (public schema, 8 tables)

- All pages are "use client" components.

- app/layout.tsx wraps the whole app in <AuthProvider>; a hydrated flag prevents rendering

until localStorage auth state has been read.

- Every protected page has a **double guard**:

1. useEffect → router.push("/authenticate") if not authenticated (or "/" if not admin).

2. Render-time if (!authenticated) return null (and admin pages additionally if

(user?.role !== "admin") return null).

- components/Header.tsx is a client component that reads useAuth() directly, so admin-only

controls (Daily Report, Users, Lists) can be conditionally rendered.

Routes

Route	Page	Access	Purpose
/authenticate	Login	public	Username/password sign-in
/	Dashboard	any logged-in user	Admission-No + Name search, student cards, add-record modals
/student/[admissionNo]	Student detail	any logged-in user (read); admin (write)	Full history + add strikes/black marks/gold marks/punishments/comments

Route	Page	Access	Purpose
/discipline	Records	any logged-in user (read); admin (status toggle)	Unified feed of all record types with tabs/filters/sort
/lists	Lists overview	admin only	Browse/search/sort lists, active/inactive
/lists/create	Create list	admin only	Build a list by admission number
/lists/[listId]	List detail	admin only	Add/remove students, bulk strikes/black marks
/users	User management	admin only	CRUD on the users table

Shared components

- **`Header`** — indigo gradient bar: brand icon (/ICON.jpeg), title, admin-only actions (Generate Daily Report button, Users, Lists), always-visible Records + Dashboard links, logged-in username, and a red-pill **Logout** button.
- **`Student`** — reusable result card: name (links to detail page), class + house, amber/rose/emerald count boxes for strikes/blackmarks/goldmarks, and 4 action buttons. Accepts `showActions` prop (default `true`) — the dashboard passes `user?.role === "admin"` so view-only users get read-only cards.
- **`Modal`** — thin wrapper around the native `<dialog>` element (`showModal()/close()`), locks body scroll while open, used by every form on every page.
- **`GenerateDailyReportButton`** — admin-only PDF export (see §7).

4. Data Storage — Database Schema

Backend: **Supabase Postgres**, project ref kjpvfhcbnehcmxyzpurk, public schema. **8 tables** exist

(verified by live probe; row counts current as of this report).

Column types marked * are inferred from actual API responses (PostgREST JSON) rather than a stored DDL document.

4.1 students — the roster (2,746 rows)

Column	Type	Notes
Admission No	integer	Primary key / natural key, e.g. 24781
Name with Initials	text	e.g. "THOTAGAMUWA JU"
Grade	text	e.g. "COLL SCI A" (academic grade)
Class	text	e.g. "COLL SCI A 1"
School House	text	e.g. "STONE"

Sample row: { "Admission No": 24781, "Grade": "COLL SCI A", "Class": "COLL SCI A 1",
"Name with Initials": "THOTAGAMUWA JU", "School House": "STONE" }

4.2 strikes — negative discipline records (16 rows)

Column	Type	Notes
id	integer (PK)	auto
created_at	timestamp _{tz}	defaults to now(); timezone-aware UTC
Admission No	integer	FK-ish to students (no DB constraint)
Category	text	one of the strike category keys (see §5)

Sample row: { "id": 6, "created_at": "2026-07-18T12:55:03+00:00", "Admission No":
22412, "Category": "grooming" }

4.3 blackmarks — serious negative records (12 rows)

Column	Type	Notes
id	integer (PK)	auto
created_at	timestampz	defaults to now()
Admission No	integer	
Reason	text	category key (see §5)
issuedBy	text	free-text name of the prefect who issued it

Sample row: { "id": 2, "created_at": "2026-07-18T12:55:15+00:00", "Admission No":

22412, "Reason": "late", "issuedBy": "hasintha" }

4.4 goldmarks — positive records (11 rows)

Same shape as blackmarks: id (PK), created_at, Admission No, Reason (gold-mark category key, see §5), issuedBy.

4.5 comments — free-text notes (7 rows)

Column	Type	Notes
id	integer (PK)	auto
created_at	timestampz	defaults to now()
Admission No	text*	returned as a string by the API (the app writes/reads numbers; PostgREST casts)
commentor	text	free-text name
commentText	text	the comment body

Sample row: { "id": 1, "created_at": "2026-07-30T16:08:20+00:00", "Admission No":

"22412", "commentor": "Hasintha", "commentText": "... " }

4.6 punishments — assigned punishments (1 row)

Column	Type	Notes
id	integer (PK)	auto
created_at	timestampz	defaults to now()
Admission No	integer	
Type	text	detention / weekend-duty / cleanup / other
Reason	text	optional description
assignedBy	text	free-text name
Status	text	"ongoing" (default) or "completed"

Sample row: { "id": 4, "Admission No": 22412, "Type": "other", "Reason": "...",
"assignedBy": "dihen", "Status": "ongoing" }

4.7 users — authentication accounts (2 rows)

Column	Type	Notes
id	bigint identity (PK)	auto
username	text	UNIQUE
password	text	bcrypt hash (never plaintext)
role	text	'admin' or 'view-only' (CHECK constraint)
created_at	timestampz	defaults to now()

Sample row: { "id": 2, "username": "prefects", "password": "\$2b\$10\$PddTJBa...",
"role": "admin", "created_at": "2026-08-10T10:17:47+00:00" }

DDL (as provided to the user earlier):

```
CREATE TABLE users (

  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

  username TEXT NOT NULL UNIQUE,

  password TEXT NOT NULL,          -- bcrypt hash

  role TEXT NOT NULL DEFAULT 'view-only' CHECK (role IN ('admin', 'view-only')),

  created_at TIMESTAMPTZ DEFAULT now()

);
```

4.8 lists — named student groups (4 rows)

Column	Type	Notes
id	integer (PK)	auto
title	text	e.g. "Detention"
students	integer[] (array)	array of admission numbers — no join table; membership is denormalized into this array
active	boolean	active/inactive toggle
createdBy	text	free-text creator name
created_at	timestamptz	
updated_at	timestamptz	maintained on update

Sample row: { "id": 1, "title": "Detention", "students": [26622, 22412, 26621],
 "active": true, "createdBy": "Hasintha", "created_at": "2026-07-30T16:45:19+00:00",
 "updated_at": "2026-07-30T16:45:19+00:00" }

A `list_students` table was proposed in early specs but does **not** exist — list membership lives in the `students` array column.

5. Category Enums (stored as text keys, displayed via labels)

Shared in `lib/labels.ts` (`categoryLabels`), with a duplicate copy still inline in

`app/student/[admissionNo]/page.tsx`.

Key	Display label	Used in
grooming	Personal Grooming	strikes, blackmarks
repeated-punish	Repeated Punishments	strikes, blackmarks
bullying	Bullying	strikes, blackmarks
late	Getting Late Often	strikes, blackmarks
substances	Substances	strikes, blackmarks
classfuckup	Classroom Behavior	strikes, blackmarks
clubbing	Clubbing	strikes, blackmarks
good-behavior	Good Behavior	goldmarks
giving-back	Giving Back to College	goldmarks
excellent-academics	Excellent Academics	goldmarks

Punishment types (`punishmentLabels`, inline in two pages): `detention` → Detention, `weekend-duty` → Weekend Duty, `cleanup` → Cleanup, `other` → Other.

Categories actually present in production data: `strikes` → grooming, late, repeated-punish; `blackmarks` → late, grooming, repeated-punish, classfuckup; `punishments` → other. The dropdowns also offer categories that have never been used (e.g. bullying, substances, clubbing).

6. How Data Is Handled

All reads/writes are **direct browser → Supabase** calls. No server-side data handling exists.

Read patterns

- **Admission No tab (dashboard):** one students query filtered by admission number, then three separate count queries against strikes, blackmarks, goldmarks for that student.

- **Name tab (dashboard):** first search lazily fetches **all 2,746 students** (cached in a useRef)

and **all rows of the 3 record tables** to build a { admissionNo: { strikes, blackmarks, goldmarks } } counts map (also cached in a ref). Subsequent searches reuse the caches; counts are refreshed after any insert.

- **Name matching** (lib/nameSearch.ts): both query and stored name are normalized

(uppercase, strip everything non-alphanumeric) and matched by **substring** (e.g. de silva

matches DE SILVA ES). Requires ≥ 2 normalized characters; results sorted alphabetically and

paginated 20 at a time ("Load more").

- **Student detail page:** 6 parallel-ish sequential queries (student + 5 record tables), all filtered by

Admission No.

- **Records page (`/discipline`):** fetches all 5 record tables + the full student list in parallel (

Promise.all), builds a **unified record feed** (UnifiedRecord[]) normalizing

strikes/blackmarks/goldmarks/punishments/comments into one shape, then filters (date range,

issuer substring, student name/admission substring, per-tab category/type) and sorts

(newest/oldest/category) **entirely client-side**.

- **Lists:** overview fetches all lists once; detail page fetches the list then resolves its students array via `.in("Admission No", list.students)`.

Write patterns

- Every insert/update/delete is an `await supabase.from(table).insert/update/delete(...)` inside a modal's Save handler, followed by re-fetching the affected data (`fetchData()` / `refreshRecordCounts()`).
- Strikes insert `{ "Admission No", Category }`; black marks & gold marks additionally issuedBy; punishments insert `{ Type, Reason, assignedBy, Status: "ongoing" }`; comments insert `{ commentor, commentText }`.
- **Bulk operations:** from list detail, selecting students enables bulk insert of an array of strike/blackmark rows in one call.
- **Punishment status** is toggled with `update({ Status: "completed" | "ongoing" })`.
- Errors are surfaced with `alert()` + `console.error`; most calls don't check errors (strikes, blackmarks, comments, list ops), while goldmarks/punishments/status toggles do.

Timezone handling

Record timestamps are UTC timestamp_{tz}. The daily report computes "today" in the **user's local timezone** by building new `Date(y, m, d) / +1` day boundaries and converting to ISO (UTC) for `gte/lt` range filters.

7. The Daily Report (PDF)

- **Entry point:** admin-only white pill button in the header (`components/GenerateDailyReportButton.tsx`), with a loading state and a mobile-only short label.
- **Data:** on click, fetches today's strikes and blackmarks (local-timezone day boundaries, newest first) plus the full student roster in parallel.
- **Empty day:** `alert("No strikes or blackmarks recorded today.")` and no PDF is produced.
- **Rendering:** the heavy PDF libraries (`jspdf`, `jspdf-autotable`) are **dynamically imported on demand** so they never inflate the initial bundle. Layout: title → date → summary line ("Total: N strikes, N blackmarks") → divider → per-section (only sections with records): an indigo `Strikes (N)` / `Blackmarks (N)` header, then per-student blocks — a grid-style autotable whose header row is `name | Adm: # | Class: ... | House: ...` and body rows are the category labels of that student's records. Page breaks are handled by tracking `lastAutoTable.finalY`.

- **Download:** `doc.save("daily-report-YYYY-MM-DD.pdf")` triggers an automatic browser

download (no user interaction beyond the click).

- Records are grouped by admission number (order of first appearance = newest first).

8. Authentication & Authorization

8.1 How login works (`lib/AuthContext.tsx`)

1. The `/authenticate` form calls `login(username, password)` (async).
2. `login` queries the **users** table: `select("id, username, password, role").eq("username", username).maybeSingle()`.
3. If a row exists, `bcryptjs.compareSync(password, storedHash)` runs in the browser (`bcryptjs` is dynamic-imported and cached).
4. On match: `setUser({ id, username, role })`, `setAuthenticated(true)`, and the **user object is persisted to `localStorage` under the key `"user"`**.
5. Wrong username or wrong password → returns `false` → the form shows `*"Invalid username or password"`.
6. On app mount, `AuthProvider` rehydrates from `localStorage`; children aren't rendered until hydration finishes (avoids a login-screen flash).

7. **Logout** (red pill in header) clears state + localStorage and redirects to /authenticate.

The original hardcoded admin/password fallback was **removed** — login now works only against the users table.

8.2 Roles

- **`admin`** — full access to everything: add strikes/black marks/gold

marks/comments/punishments, toggle punishment status, bulk actions, lists (all 3 pages), user management, daily report, and the admin-only header links (Users, Lists, Generate Daily Report).

- **`view-only`** — read-only access: dashboard search, student detail (viewing history only), and

the Records page (viewing only). **All** write controls are hidden: the Student card action buttons (

showActions={false})), the detail-page action buttons and punishment status toggles, the

Records-page status toggles, the bulk-action buttons in lists, the Generate Daily Report button, the

Users link, the Lists link, and the /users + /lists pages themselves (page-level guards redirect to

/ and render nothing).

8.3 User management (/users, admin only)

- Add user (username, password → bcrypt-hashed with bcryptjs before insert, role select).
- Edit user (username, optional new password, role).

- Delete user — guarded: **you cannot delete yourself**, and **the last admin cannot be deleted**

.

- Duplicate usernames are caught via the DB unique constraint and reported as "Username

already exists".

- Password fields in the admin UI are plain `<input type="text">` (visible), so the admin can see

what they typed.

8.4 Permission matrix

Capability	Admin	View-only
Search students / view profiles	✓	✓
Add strike / black mark / gold mark / comment	✓	✗ (hidden)
Add / complete punishments	✓	✗ (hidden)
Generate Daily Report (PDF)	✓	✗ (hidden)
Lists (view, create, bulk actions)	✓	✗ (link + pages hidden)
User management (/users)	✓	✗ (link + page hidden)
Logout	✓	✓

9. Security Considerations & Known Limitations

These are important context for a school system holding student discipline records:

1. **No server-side security.** The Supabase **publishable/anon key is embedded in the client**

bundle and there are no RLS policies in play that the app relies on. Anybody who can view the

page source can call the Supabase REST API directly to read or modify any table (the same operations the UI performs). The login system is a **UI gate, not an access-control boundary**.

2. **Role impersonation.** Because the "session" is just a `localStorage` JSON object (`{id, username, role}`) with no server verification, a user could manually edit `localStorage` to "role": "admin" and gain admin UI on the next reload. (Server-side data protection would require real Supabase Auth + RLS.)

3. **Passwords are hashed** (bcrypt) at rest, which is good — but the bcrypt comparison runs **in the browser**, meaning the hash comparison logic ships to the client (acceptable for this app, unusual for production auth).

4. **No audit trail.** `issuedBy` / `assignedBy` / `commentor` are free-text fields, not references to `users.id` — anyone can type any name.

5. **No referential integrity.** Record tables reference `students`. "Admission No" by convention only (no foreign keys), and `comments`. "Admission No" is a text column.

6. **Data volumes are fetched wholesale** — the dashboard name tab and Records page pull all students (~2,700) and all records into the browser; fine at current size, but won't scale past low tens of thousands of rows.

7. **Minor inconsistencies:** `categoryLabels` is duplicated (shared in `lib/labels.ts`, plus a local

copy in the student detail page); the "he lied to you" easter-egg copy exists in the not-found states;

several inserts ignore Supabase errors.

10. Project Structure

web/

└─ app/

| └─ layout.tsx # Root layout, AuthProvider wrapper, Geist fonts

| └─ globals.css # Tailwind v4

| └─ page.tsx # Dashboard (admission + name search, student cards, modals)

| └─ authenticate/page.tsx # Login page (branding icon + form)

| └─ discipline/page.tsx # Records: unified feed, tabs, filters, sorting

| └─ student/[admissionNo]/page.tsx # Student detail (history + add modals)

| └─ users/page.tsx # Admin user management (CRUD)

| └─ lists/

| └─ page.tsx # Lists overview (search/filter/sort/toggle)

| └─ create/page.tsx # Create list

| └─ [listId]/page.tsx # List detail (add/remove, select mode, bulk actions)

└─ components/

| └─ Header.tsx # Client header: nav, admin controls, logout

| └─ Student.tsx # Result card (counts + action buttons, showActions prop)

| └─ Modal.tsx # <dialog> wrapper

| └─ GenerateDailyReportButton.tsx # PDF export

└─ lib/

| └─ supabase.ts # Supabase client (URL + publishable key)

| └─ AuthContext.tsx # Auth provider: user, login, logout, localStorage

| └─ nameSearch.ts # Name normalization + substring search

| └─ labels.ts # Shared category → display-label map

└─ public/ICON.jpeg # Brand icon (login + header)

11. Development

```
npm install      # install dependencies

npm run dev      # start dev server on http://localhost:3000

npm run build    # production build (validated – passes)

npm run lint     # eslint (pre-existing warnings only)
```

Supabase credentials live in `lib/supabase.ts` (project URL + publishable key). The users table and

seed admin must exist in Supabase or nobody can log in.